國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab02

姓名：李品融

學號：113310217

Lab02-Ex1
這題在計算使用 for 迴圈排列陣列所需的步數。
Code

```
int findMax(const vector<int>& arr) {
// TODO: Add counts for initialization

 int stepCount = 0 //假設步數

 int max = arr[0];
 stepCount++; //陣列存取 arr[0]與賦值給 max，1 步
// TODO: Add counts for loop operations

 stepCount++; //for 迴圈初始化(i=1)，1 步
for (int i = 1; i < arr.size(); i++) {
// TODO: Add counts for comparison and assignment

 stepCount++; //比較 i<arr.size()，1 步

 stepCount++; //比較 arr[i]>max，1 步
 if (arr[i] > max) {
  max = arr[i];
  stepCount++; //賦值運算(將新最大值存入 max)，1 步
 }
 stepCount++; //增量運算(i++)，1 步
}
// TODO: Add counts for return

stepCount++; //for 迴圈結束時最後一次失敗的比較，1 步
stepCount++; //return 動作，1 步
return max;
}
// Total operations(總運算次數):
// 2 (初始化) + 4 * (n-1) (for 迴圈內的運算) + 1 (跳出迴圈) + 1 (return 回傳結果)
// = 4 + 4(n-1) operations
// = 4n operations
// Therefore, O(n) complexity
```

Lab02-Ex2
這題在計算使用 for 雙重迴圈時，所需要的步數。
Code

```
void printPairs(const vector<int>& arr) {
// TODO: Add counts for outer loop initialization
 int stepCount = 0; //假設步數

 stepCount++; //for 迴圈初始化(i=0)，1 步
 for (int i = 0; i < arr.size(); i++) {
  // TODO: Add counts for inner loop
  stepCount++; //外層 for 迴圈比較 (i<arr.size()) ，1 步

  stepCount++; //內層 for 迴圈初始化(j=i+1)，1 步
  for (int j = i + 1; j < arr.size(); j++) {
   // TODO: Add counts for printing operation
   stepCount++; //內層 for 迴圈比較 (j<arr.size())

   cout << arr[i] << "," << arr[j] << endl;
   stepCount=stepCount+3; //使用 arr[i]+使用 arr[j]+列印出來，共三步

   stepCount++; //內層迴圈遞增(j++)，1 步

  }
  stepCount++; //j 退出時，內層迴圈最後一次失敗的比較，1 步

  stepCount++; //外層迴圈遞增(i++)，1 步
 }
 stepCount++; //i 退出時，外層迴圈最後一次失敗的比較，1 步

 cout << "Step count: " << stepCount << endl;
}
```

Lab02-Ex3

這兩題皆在進行陣列的讀取與比較，並計算所需要的步數。

Code

(a)

```
void copyArray(const vector<int>& source,
vector<int>& dest) {

 int stepCount = 0; //假設步數

 stepCount++; //for 迴圈初始化(i=0)，1 步

 for (int i = 0; i < source.size(); i++) {
  stepCount++; //比較 i<source.size()，1 步

  dest[i] = source[i];
  stepCount = stepCount + 2; //讀取 source[i]並賦
值給 dest[i]，2 步

  stepCount++; //迴圈遞增(i++)，1 步
 }

 stepCount++; //for 迴圈結束時最後一次失敗的比較，
1 步

 cout << "Step count: " << stepCount << endl;
}
// Total operations (總運算次數):
// 1 (初始化) + 4 * n (for 迴圈內運算) + 1 (跳出迴
圈)
// = 4n + 2 operations
// Therefore, O(n) complexity (線性時間複雜度)
```

(b)

```
int countElement(const vector<int>& arr, int
target) {
 int stepCount = 0; //假設步數
 int count = 0;

 stepCount++; //初始化 count=0，1 步
 stepCount++; //for 迴圈初始化(i=0)，1 步

 for (int i = 0; i < arr.size(); i++) {

  stepCount++; //比較 i<arr.size()，1 步
  stepCount++; //比較 if 判斷式(arr[i] ==
target)，1 步
  if (arr[i] == target) {
   count++;
   stepCount++; //count 遞增，1 步
  }
  stepCount++; //迴圈遞增(i++)，1 步
 }

 stepCount++; //for 迴圈結束時最後一次失敗的比較，
1 步
 stepCount++; //return 動作，1 步

 cout << "Step count: " << stepCount << endl;
 return count;
}
// Total operations (總運算次數):
// 2 (初始化) + 4 * n (for 迴圈內運算) + 1 (跳出迴
圈) + 1 (return 回傳結果)
// = 4n + 4 operations
// Therefore, O(n) complexity
```

Discussion Section
2. Why do we drop constants in Big-O notation? o If an algorithm takes 3n+2 operations, why do we say O(n)? o What's the practical importance of this? 討論結果:因為 Big-O 重點在增長階數，當資料量 n 極大時，常數項的影響將微乎其微，可以直接省略。在實務上，可以簡化複雜度，讓工程師可以更好判斷與比較演算法的好壞。 3. Looking at the examples above: o Which has more actual operations: findNumber or sumArray? o Does this change their Big O classification? Why or why not? 討論結果:sumArray 的運算步驟會比 findNumber 更多。並且這不會改變他們 Big O 的分類，因為兩者最高次項都是 n 的一次方，所以都屬於 O(n)，且執行時間都與 n 呈線性比例關係。